PIC 10A 1A

TA: Bumsu Kim

This content is protected and may not be shared, uploaded, or distributed.

The author does not grant permission for these notes to be posted anywhere without prior consent.

UCLA

Today… Review

- Variables
- Control Flow
- Random Numbers
- Functions
- Classes
- Vectors, Arrays, and Pointers
- Exercise Problems

Variables

Basics

Integral and Floating-Point Types

Strings

Variables

- C++ is case sensitive. For example,

```
#include <iostream>
using namespace std;

int Main() {
        cout << "Hello, World!" << endl;
        return 0;
}
```

will not compile

UCLA

Variables

- C++ is case sensitive. For example,

```
#include <iostream>
using namespace std;

int Main() {
        cout << "Hello, World!" << endl;
        return 0;
}
```

will not compile. Neither will

```
#include <iostream>
using namespace std;

int main() {
        int num = 2;
        cout << Num << endl;;
        return 0;
}
```

UCLA

Variables

- What is a variable?

```
#include <iostream>
using namespace std;

int main() {
    int my_number;
    my_number = 5;
    cout << "The content of my_num is ";
    cout << my_number << "." << endl;
    return 0;
}
```

- The output is:

```
C:\Windows\system32\cmd.exe
The content of my_num is 5.
Press any key to continue . . .
```

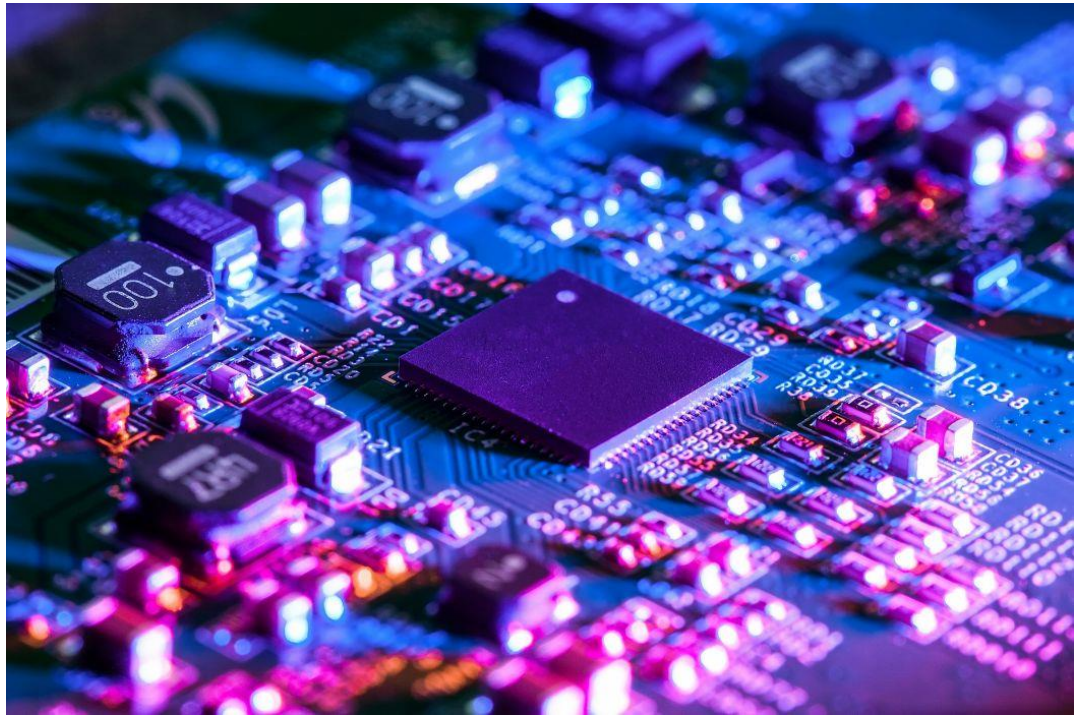
UCLA

Types

- Computers use **bits** to store and process data
  - 8 bits = 1 byte (usually the smallest unit for this course)

- Bit
  - Binary digit
  - 0 or 1
- Byte
  - 8 bits

On (1) / Off (0) (electric signals)



Types

- Computers use **bit**s to store and process data
  - 8 bits = 1 byte (usually the smallest unit for this course)

- Bit
  - Binary digit
  - 0 or 1
- Byte
  - 8 bits

On (1) / Off (0) (electric signals)

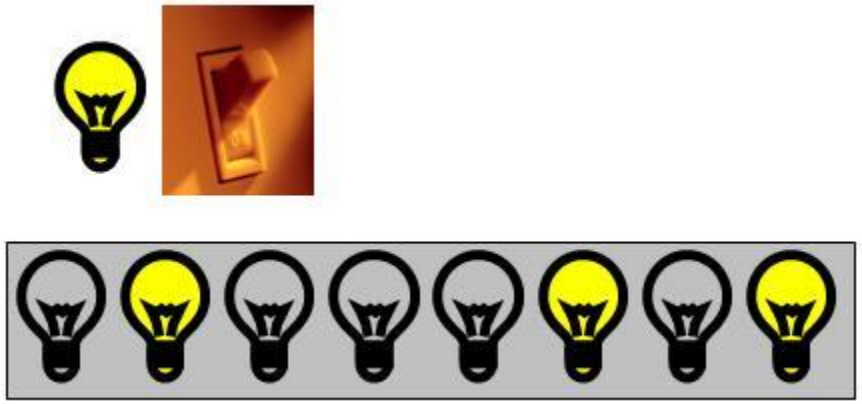
- Naturally corresponds to binary numbers
  - 01000101 (base 2)
- Floating point numbers, strings, and even graphics are essentially represented as combinations of bits

UCLA

Types

- Everything (a floating point number, a string, and even a graphical component) is essentially represented as combinations of bits in your computer

- Bit
  - Binary digit
  - 0 or 1
- Byte
  - 8 bits



On (1) / Off (0)

- So [01000101…] might mean an integer number, a floating point number, a character (like ‘a’), or even a part of an image or video.

→ You need to specify a “**type**” of the data so that the machine can interpret it as you intend

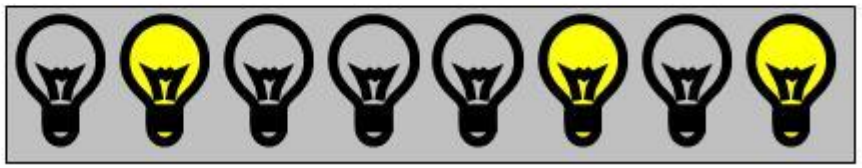
UCLA

Integral types (will be revisited next week)

- Integral type: uses base 2 representation of integers (binary numbers)
  - 8, 16, 32 or 64 **bit**s
  - = 1, 2, 4 or 8 **byte**s
  - ex) 0100 1111 0101 0000 (base 2)
  - $= 2^{14} + 2^{11} + 2^{10} + 2^{9} + 2^{8} + 2^{6} + 2^{4} = 20304$

- Q) How many different states can be represented by 8 bits?

- Bit
  - Binary digit
  - 0 or 1
- Byte
  - 8 bits



On (1) / Off (0)

UCLA

Integral types (will be revisited next week)

- Integral type: uses base 2 representation of integers (binary numbers)
  - 8, 16, 32 or 64 **bit**s
  - = 1, 2, 4 or 8 **byte**s
  - ex) 0100 1111 0101 0000 (base 2)
  - $= 2^{14} + 2^{11} + 2^{10} + 2^9 + 2^8 + 2^6 + 2^4 = 20304$

- If you use 16 bits (= 2bytes), you can represent total 2^{16} different numbers
  - If you start from 0, then from 0 to $2^{16} - 1$
  - If you want to include negative numbers, with 0 at the middle, then -2^{15} to $2^{15} - 1$

- The type `int` and `unsigned int` usually have 4 bytes (about 4 billion states)
- For larger/smaller numbers, use `(unsigned) long long int`

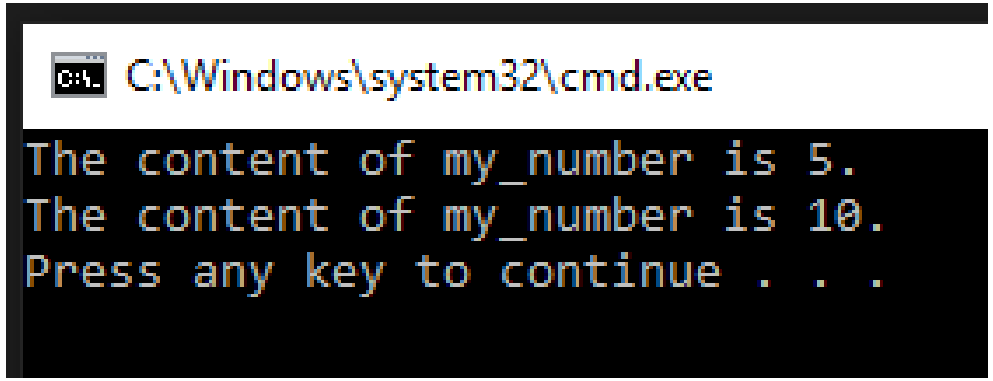
UCLA

Variables

```
#include <iostream>
using namespace std;

int main() {
    int my_number;
    my_number = 5;
    cout << "The content of my_number is " << my_number << "." << endl;
    my_number = 10;
    cout << "The content of my_number is " << my_number << "." << endl;
    return 0;
}
```

- What is the output?



UCLA

Variables – Declaration

- Variable declaration/definition:

```
int my_number;
```

  - This introduces `my_number` as a *variable*
  - Its *type* is `int`, short for integer
  - A variable is a place to store information
  - In this case `my_number` is a place to store an integer

UCLA

Variables – Declaration

```
int my_number;
```

- Q: So, is `my_number` a number?

- A: Not really.
  - `my_number` is not a number, it's a variable
  - A number is fixed. `my_number` is a container to store a number
  - `my_number` is not (an arbitrary) number because it can't store numbers like 3.2 or π. It can only store integers

UCLA

Variables – Assignment

```
my_number = 5;
```

- This writes the value 5 into `my_number`.
- The = symbol is somewhat misleading. Something like

```
my_number <- 5;
```

  would be more clear (but this doesn't work)

- Again, strictly speaking, `my_number` is not 5, but "The (current) value of `my_number` is 5"
- The first assignment is also called *initialization*

Q: What value is stored in "`my_number`" before the first assignment?

A: No one knows. Accessing an uninitialized variable results in an "**undefined behavior**" and *anything can happen*. It's considered bad! Programmers don't want a situation that they can't control.

UCLA

Variables – Assignment

- Note that, in math, it would be wrong to write

$$x = 5$$
$$x = 10$$

- We can't assert x is 5 and then assert it is 10.

- On the other hand, in programming, there's nothing wrong about

```
my_number = 5;
my_number = 10;
```

- Again, the = symbol in C++ does not mean *equality*

UCLA

Useful digression: programming tip

I don't know what's going on. This is uncomfortable!

- Move on and first try to get things to work.
  (Sometimes, e.g. when studying math, this is a terrible attitude.)
- Just because there is a part you do not understand, doesn't mean you should stop to figure it out.
- First make things work, and then worry about other things.
- In serious programming, you'll never understand everything.

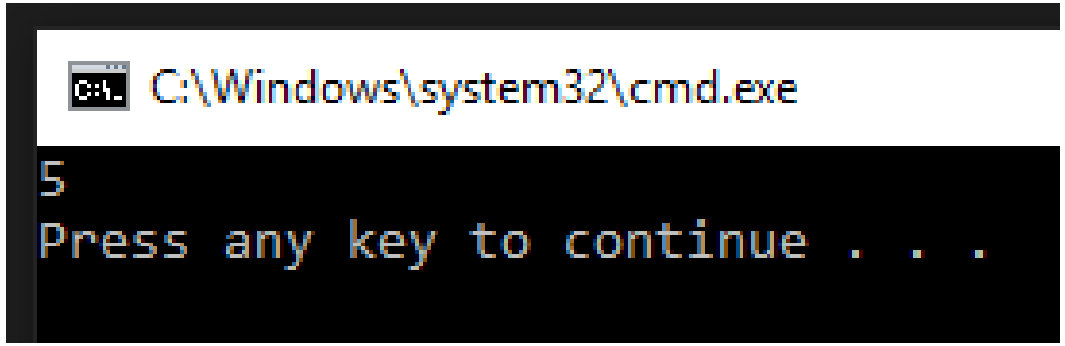
Slide from PIC10A (2017 Fall) taught by Prof. Ernest Ryu

The type `int` – operators

- Operators such as +, -, *, / are defined for the type `int`

```
int num1 = 4; // declartion & initialization at the same time
int num2 = 1;
int num3;
num3 = num1 + num2;
cout << num3 << endl;
```

- What is the output?



The type `int` – operators

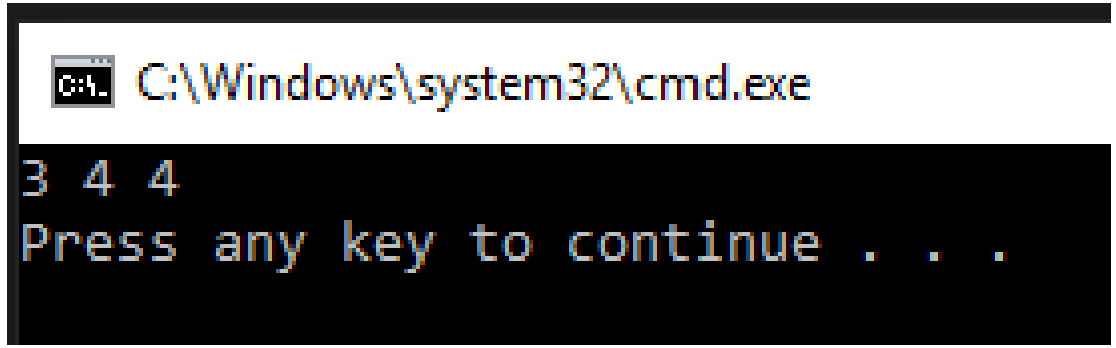
- Operators such as +, -, *, / are defined for the type `int`

```
int num4 = num1 - num2;
int num5 = num1 * num2;
int num6 = num1 / num2;
cout << num4 << ' ' << num5 << ' ' << num6 << endl;
```

Recall that:

```
int num1 = 4;
int num2 = 1;
```

- What is the output?



UCLA

The type `int` – operators

- Operators such as +, -, *, / are defined for the type `int`

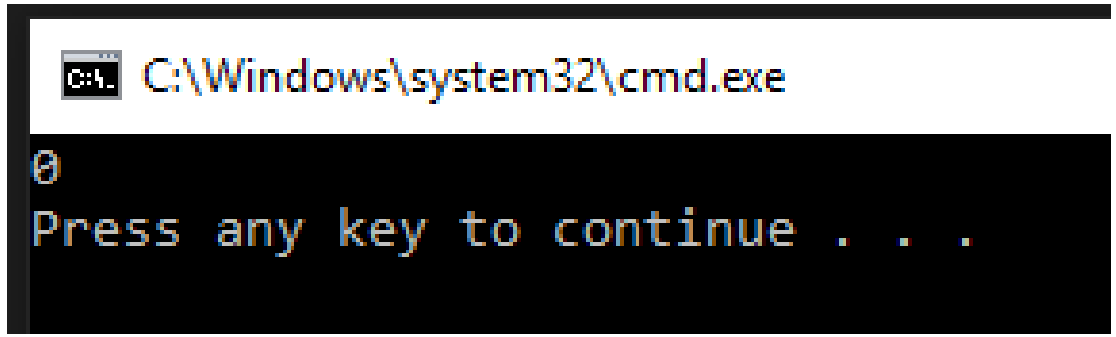
```
int num7 = num2 / num1;
cout << num7 << endl;
```

Recall that:

```
int num1 = 4;
int num2 = 1;
```

- What is the output?

Why isn't it 0.25?
More on this later…



UCLA

The type `int` – operators

- Q: So, operator "/" isn't exactly division?

- A: That's correct
  - num7 can't store 1/4 = 0.25, because it is of type `int`
  - Operator / for two `int`s performs division and take the integral part

```
int num1 = 11;
int num2 = 4;
cout << num1 / num2 << endl;
```

- What is the output?
- Use operator % for the remainder:

```
cout << num1 % num2 << endl;
```

UCLA

The type `int` – limitations

- Computers are finite. The type `int` has a maximum (and minimum) value

```
int num = 100;
cout << num << endl;
num = num * 100;
cout << num << endl;
num = num * 100;
cout << num << endl;
num = num * 100;
cout << num << endl;
num = num * 100;
cout << num << endl;
num = num * 100;
cout << num << endl;
```

- What is the output?

C:\Windows\system32\cmd.exe

```
100
10000
1000000
100000000
1410065408
-727379968
Press any key to continue . . .
```

UCLA

Integer Operations

- Recall that **+**, **-**, and ***** work as we expect from everyday math
- However, **/** for two integers does "integer division"
  - We only take the quotient, and the remainder is discarded
- On the other hand, **%** finds the remainder
- For example, 14/3 == 4 and 14%3 == 2, because 14 = 3*4 + 2
- Maybe a similar question:

```
int n = 4.567; // what is the value of n?
```

UCLA

More on `int` operators

- Special operators for integral types (for now):
  - ++, ++, --, and --
  - (Post/pre)-(increment/decrement) operators

```
int n = 3;
++n; // pre-increment
n++; // post-increment
--n; // pre-decrement
n--; // post-decrement
```

- Pre and Post operators are different (in terms of their returned values)
- In general, pre-increment/decrement is preferred because, internally, there's a non-necessary copy in post-operators

UCLA

More on operators

- Compound assignments: +=, -=, *=, /=

- Usage: x = x [operator] y;
  - [operator] can be +, *, -, / (and others)

```
int n = 3;
n += 2; // n = n + 2
n -= 2; // n = n - 2
n *= 2; // n = n * 2
n /= 2; // n = n / 2
```

- Q: What happens if you do n %= 2 when n is even/odd (respectively)?

UCLA

Floating-point Numbers

- Last time, I mentioned that the type determines how the “bits” should be read and interpreted
- For instance, we have different bits for x and y here:

```
double x = 5;
int y = 5;
```

- Let’s talk more about floating point numbers

UCLA

Floating-point Numbers

- Floating-point types: uses the "scientific notation," in base 2 numbers
  - Ex) 1902849287.4356 → 1.902×10^{9}
  - Ex) 0.0000034453234 → 3.445×10^{-6}
  - Loses precision, but more efficient for finite storage

- $\text{Mantissa} \times \text{Base}^{\text{Exponent}}$

- Of course, the Base is 2, and the mantissa and the exponent are represented in base 2 numbers

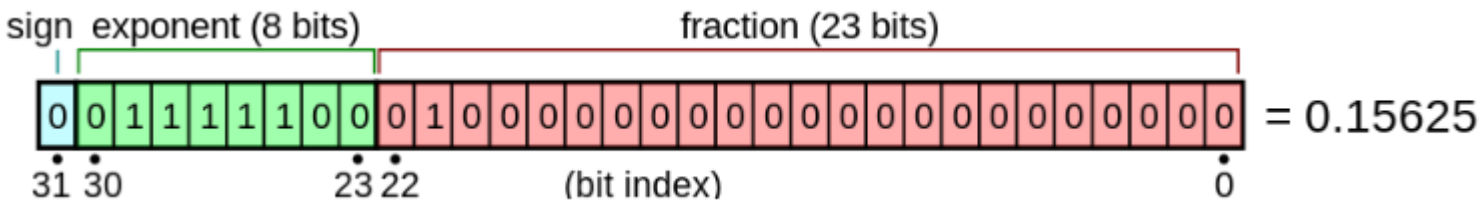
- For instance, "4.35" is stored approximately as 4.3499999999999999645
  - Example:

```
int n = 4.35 * 100;
```

UCLA

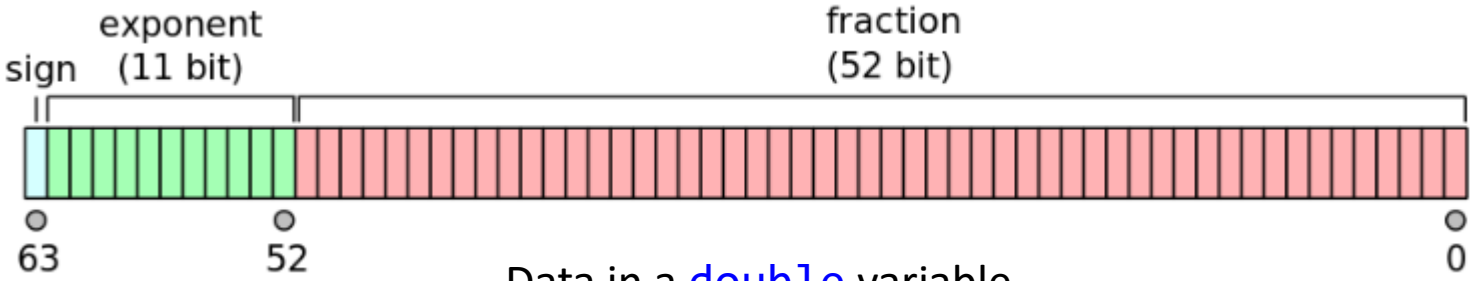
Floating-point Numbers

- Floating-point types:
  - `float`: 4 bytes
  - `double`: 8 bytes



Data in a `float` variable

- Double means "double-precision"



Data in a `double` variable

- 23-bit mantissa → about 7 significant digits in decimal (`float`)
- 52-bit mantissa → about 15 significant digits in decimal (`double`)

UCLA

The type `double` – operators

- Operators such as +, -, *, / are defined for the type `double`
- Fortunately, they are not as tricky as the case of `int`
- Use them like real number arithmetic operators, just don't forget that they have finite precision, and upper/lower limits
- There is a small number ***eps*** such that **1 + *eps* == 1** !!
  - Called a machine epsilon, and sets the limit of precision in your number system
  - Example in Live Coding

UCLA

Implicit and Explicit Casts

- Always use EXPLICIT casts

```
double x = 1;  // bad: implicit casting
int y = 1.5;   // worse: loss of information
double y = 1.0; // good
int z = static_cast<int>(1.5); // good
```

- Use `static_cast` when you should make it clear

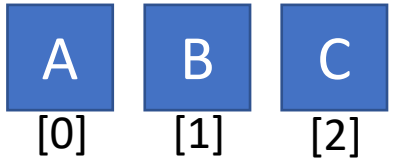
- You will see more confusing implicit typecasts later
- So let's abide by a good coding practice

UCLA

String Manipulations

- Recall the type `char`, used to store a single character
- Strings can be thought as a `char` array (i.e., a sequence of `char` variables), with some *features*

```
string str1 = "ABC";
```



Sequence of char objects
Each `char` can be accessed by the **[ ] operator** (subscript operator)
e.g. `str1[1]`

```
str1.length();
```

This expression returns the length of the string

```
str1.substr(0, 2);
```

This expression returns the substring starting from 0th position with length 2

```
str1 + "DEF";
```

This expression returns the string formed by concatenating `str1` and `"DEF"`, so the resulting string contains `"ABCDEF"`

UCLA

String Manipulations

- There are a lot of useful features defined for string
- You can find them in the lecture notes

- Good Reference: http://cplusplus.com/
  - Teaching *how* to fish is in fact more efficient
  - Right now it can be hard for you to read the documentation
  - But you will be much more comfortable at reading it by the end of this course
  - e.g. Documentation for `string`:

std::string <string>

```
typedef basic_string<char> string;
```

String class

Strings are objects that represent sequences of characters.

The standard `string` class provides support for such objects with an interface similar to that of a standard container of bytes, but adding features specifically designed to operate with strings of single-byte characters.

The `string` class is an instantiation of the basic_string class template that uses `char` (i.e., bytes) as its *character type*, with its default char_traits and allocator types (see basic_string for more info on the template).

Note that this class handles bytes independently of the encoding used: If used to handle sequences of multi-byte or variable-length characters (such as UTF-8), all members of this class (such as length or size), as well as its iterators, will still operate in terms of bytes (not actual encoded characters).

Member functions

(constructor)	Construct string object (public member function)
(destructor)	String destructor (public member function)
operator=	String assignment (public member function)

Iterators:

begin	Return iterator to beginning (public member function)
end	Return iterator to end (public member function)
rbegin	Return reverse iterator to reverse beginning (public member function)
rend	Return reverse iterator to reverse end (public member function)
cbegin C++11	Return const_iterator to beginning (public member function)
cend C++11	Return const_iterator to end (public member function)
crbegin C++11	Return const_reverse_iterator to reverse beginning (public member function)
crend C++11	Return const_reverse_iterator to reverse end (public member function)

Capacity:

size	Return length of string (public member function)
length	Return length of string (public member function)
max_size	Return maximum size of string (public member function)
resize	Resize string (public member function)
capacity	Return size of allocated storage (public member function)
reserve	Request a change in capacity (public member function)
clear	Clear string (public member function)

UCLA

Control Flow

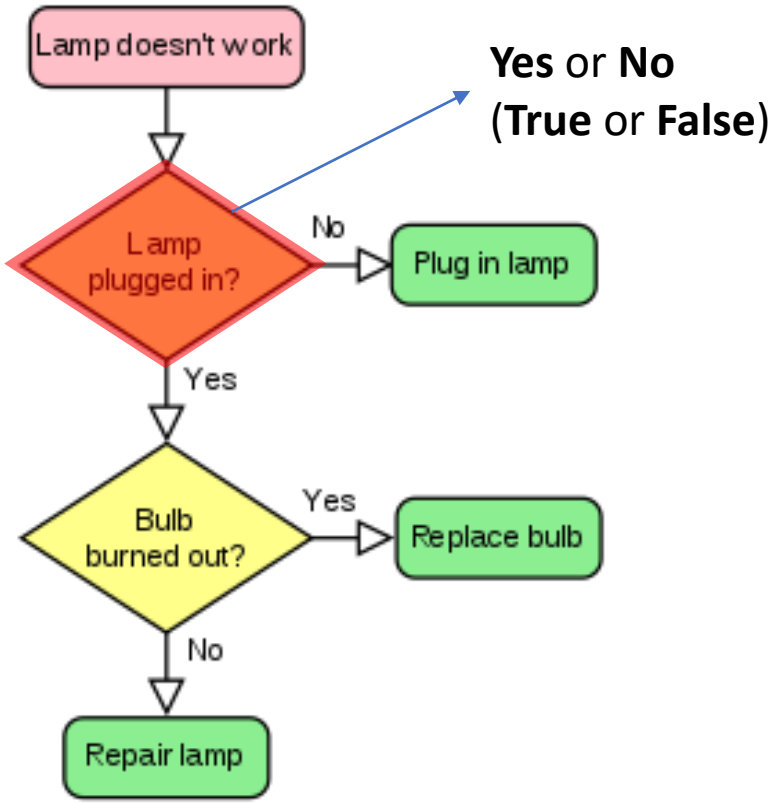
Boolean Type

Conditional Statements: if, else, and else if

Loops: for, while, and do while

Control Flow – `if` / `else` / `else if`

- Flowchart?



- Sometimes your program should run differently according to the current state
- In the world of programming, every question is basically a yes-no question
  - It's quite natural if you recall how the computer works; it's always binary. A light bulb (*bit*) can only be *on(1)/off(0)*
- More formally, the program evaluates an expression and determines if it is `true` or `false`
- Recall the "bool" (Boolean) type:

```
bool true_or_false; // bool literals: true and false
```

UCLA

The Type bool

- The type `bool` is one of the *fundamental types* in C++

- `bool` uses 1 byte (not 1 bit!) to store a logical value, "`true`" or "`false`".
  - `true` and `false` are `bool` literals

```
Microsoft Visual Studio Debug Console
1
true
```

- Comparison operators return `bool` values

```
5 < 3;
(bool)false
```

```
int a = 5, b = 3;
bool comparison = (a >= b);
cout << comparison << endl;
```

```
Microsoft Visual Studio Debug Console
1
```

```
int a = 5, b = 3;
bool comparison = (a >= b);
cout << comparison << endl;

cout << boolalpha;
cout << comparison << endl;
```

UCLA

The Type `bool`

- Every numeric value other than 0 is implicitly casted to `true`
  - 0 is false
- Other expressions may or may not be convertible to `bool`
  - More on this later…

- Just like operators +, –, /, * that are defined for the numeric types, there are operators designed for Boolean variables, called **logical operators**
  - Some of the common logical operators are NOT(unary), AND(binary), and OR (binary)
  - e.g.
    - **(NOT(** `true` **))** evaluates `false`
    - **(** `true` **AND** `false` **)** evaluates `false`, and **(** `true` **AND** `true` **)** evaluates `true`
    - **(** `false` **OR** `true` **)** evaluates `true`, etc.
  - Operator NOT : ! → ! (`true`) stands for **NOT(** `true` **)**
  - Operator AND : && → (`true && false`) stands for **(** `true` **AND** `false` **)**
  - Operator OR : || → (`false || true`) stands for **(** `false` **OR** `true` **)**

UCLA

Boolean Algebra

- Recall that
  - A && B is true only when A and B are both true
  - A || B is false only when A and B are both false
- De Morgan's law:
  - !(A && B) == (!A) || (!B)
  - !(A || B) == (!A) && (!B)

Truth Tables

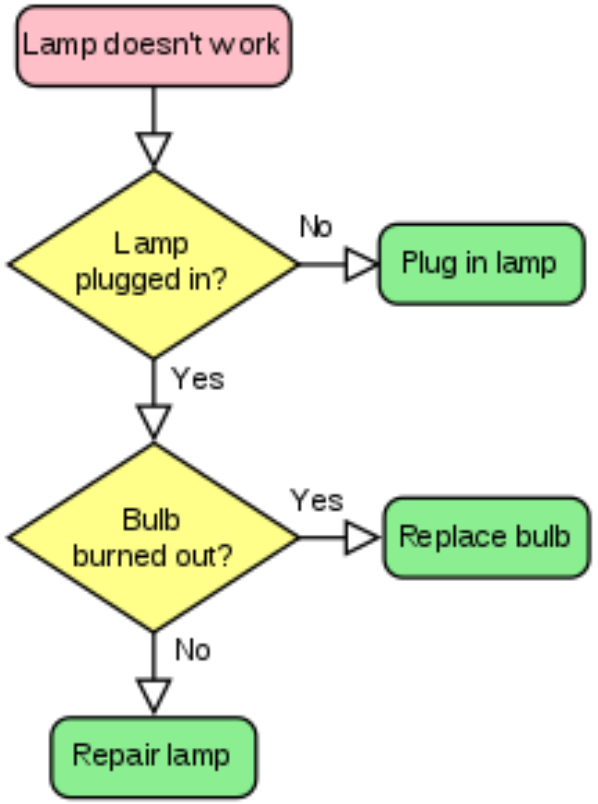
&&		P	
		T	F
Q	T	T	F
	F	F	F

||		P	
		T	F
Q	T	T	T
	F	T	F

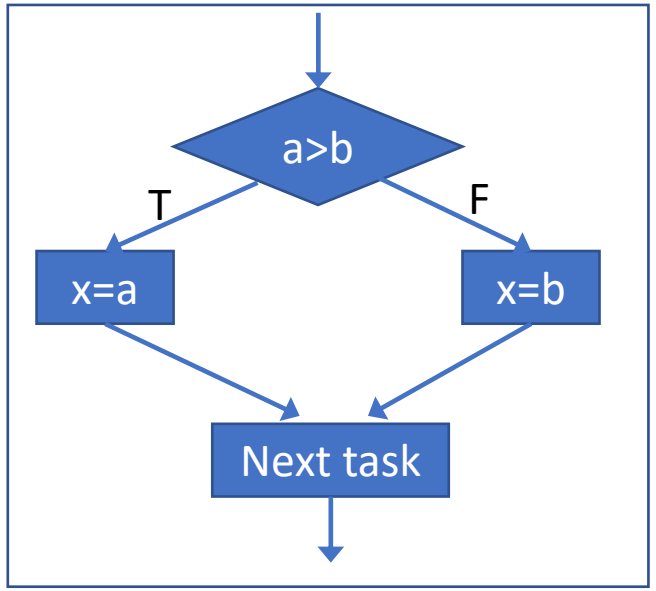
UCLA

Control Flow – `if` / `else` / `else if`

- Now you're ready to implement a flow chart like
- Only need to use "`if`" and "`else`"



- For instance, if you want `x = max(a, b)`, the flow char would look like the following:



- The implementation is simple:

```
if (a > b) { x = a; }
else { x = b; }
```

UCLA

A Useful Digression – Coding Style

- Recall that C++ is quite generous about the white spaces
- It is important, however, to follow a ***standard*** coding style for readability
- So,

```
if (a > b) { x = a; }
else { x = b; }
```

or even

```
if (a > b) x = a; else x = b;
```

Without curly braces!

is (grammatically) allowed, it is better to write it as either

```
if (a > b) {
      x = a;
} else {
      x = b;
}
```

or

```
if (a > b)
{
      x = a;
}
else
{
      x = b;
}
```

UCLA

A Useful Digression – Coding Style

- For instance, Google has its own C++ style guide:

Each of `if`, `else`, and `else if` belong on separate lines. There should be a space between the `if` and the open parenthesis, and between the close parenthesis and the curly brace (if any), but no space between the parentheses and the condition.

```
if (condition) {  // no spaces inside parentheses
  ...  // 2 space indent.
} else if (...) {  // The else goes on the same line as the closing brace.
  ...
} else {
  ...
}
```

Google C++ Style Guide: https://google.github.io/styleguide/cppguide.html#Conditionals

```
if(condition) {   // Bad - space missing after IF.
if ( condition ) { // Bad - space between the parentheses and the condition
if (condition){   // Bad - space missing before {.
if(condition){    // Doubly bad.
```

- Conclusion: Good coding style is important!

UCLA

Control Flow – `if` / `else` / `else if`

- Good coding practice: always use curly braces {} with control statements
- Even when you only need to execute a single expression

Bad

```
if (a > b)
    x = a;
else
    x = b;
```

vs

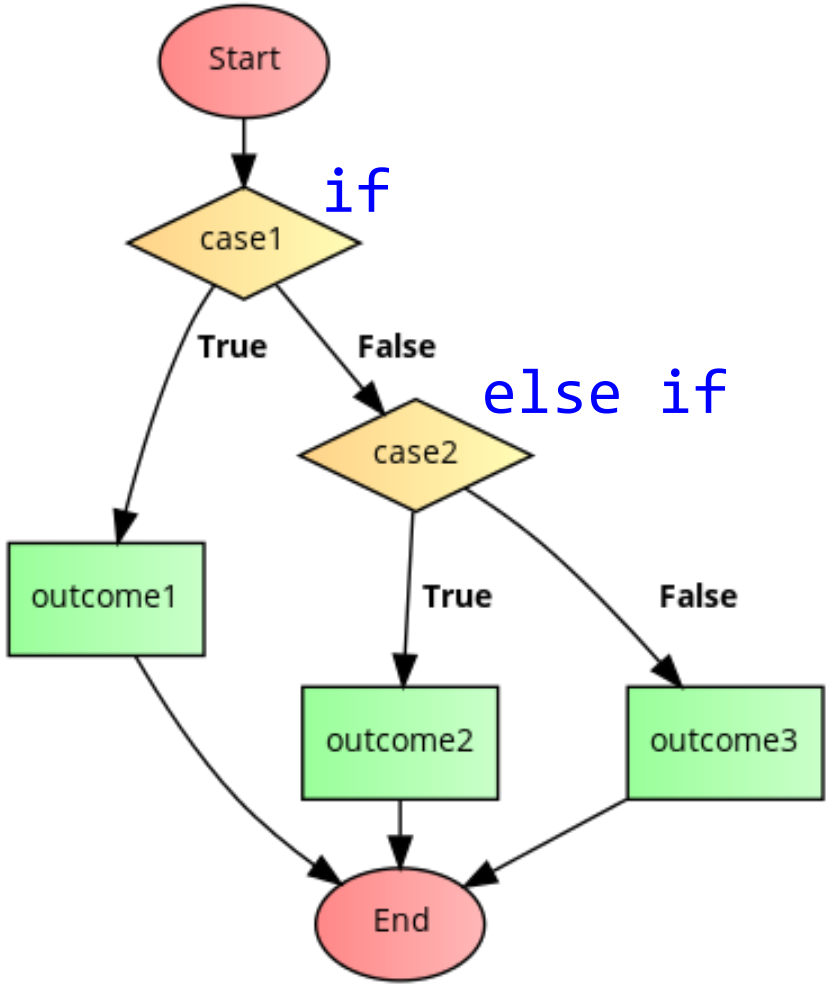
```
if (a > b) {
    x = a;
} else {
    x = b;
}
```

Good

UCLA

Control Flow – `if` / `else` / `else if`

- `else if` is just a compound form of `else{ if ( … ) }`



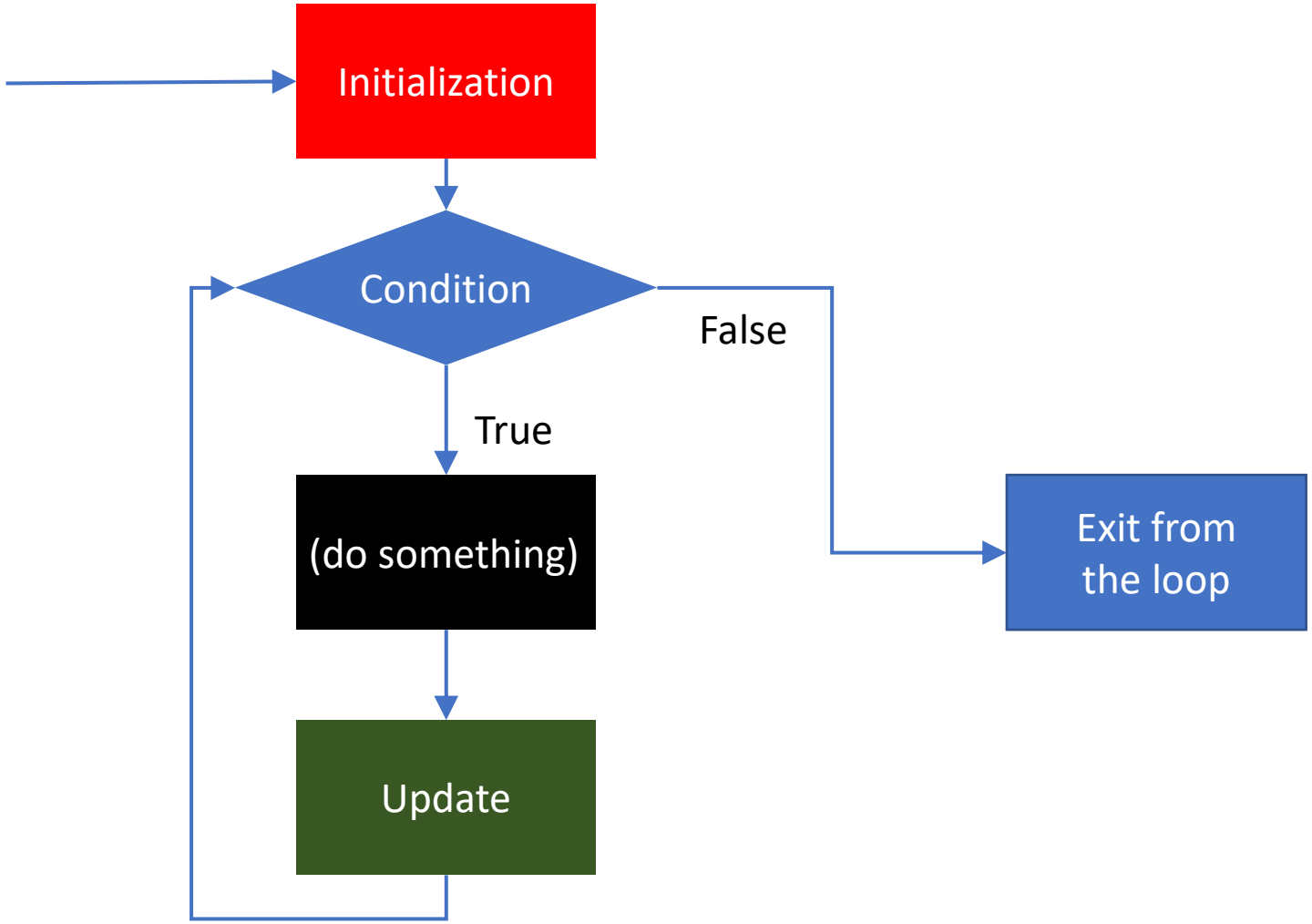
Implementation

```
if (case1) {
      outcome1;
}
else if (case2) {
      outcome2;
}
else {
      outcome3;
}
```

UCLA

Review on `for`, `while`, and `do while`

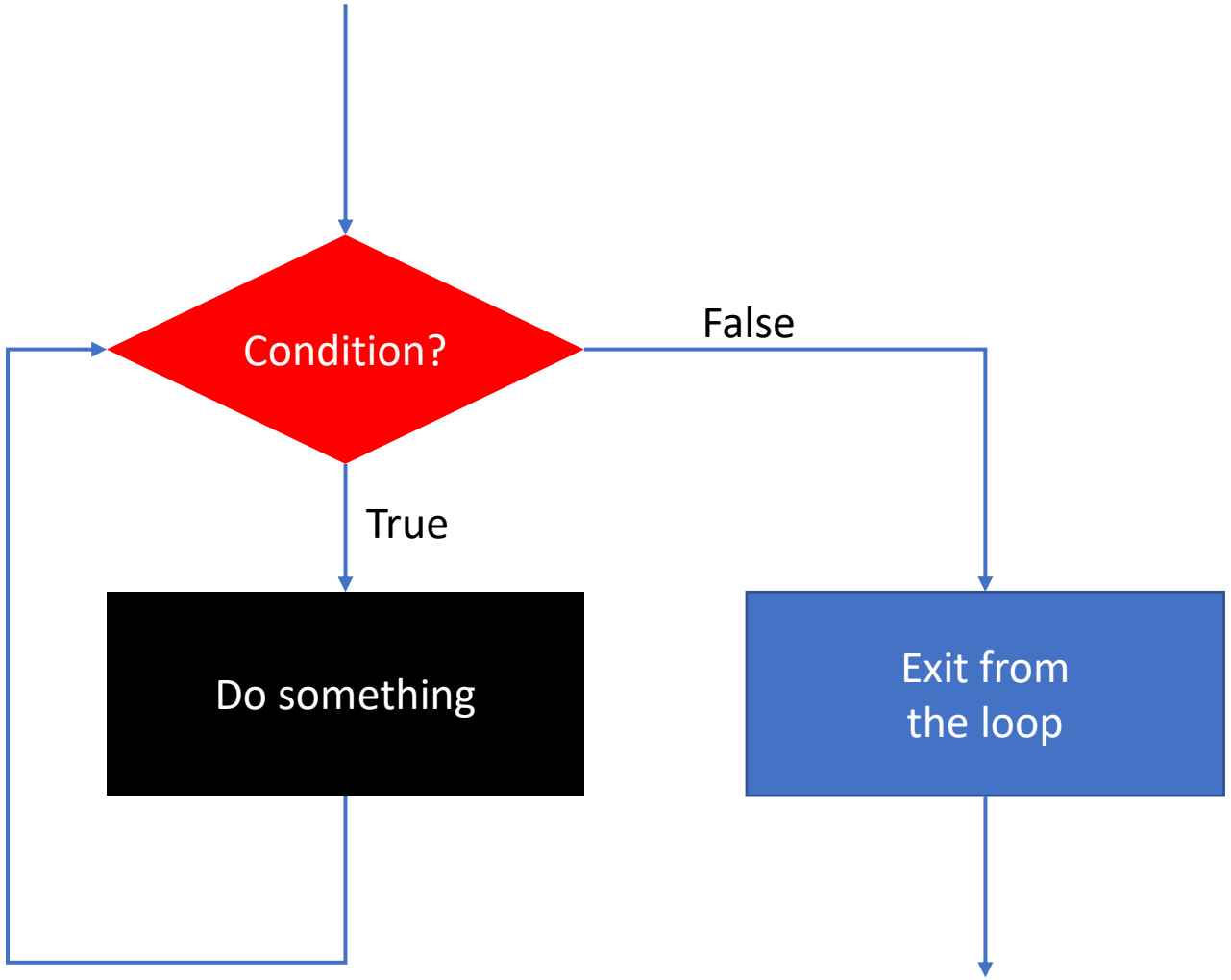
- Flow chart (for)



UCLA

Review on `for`, `while`, and `do while`

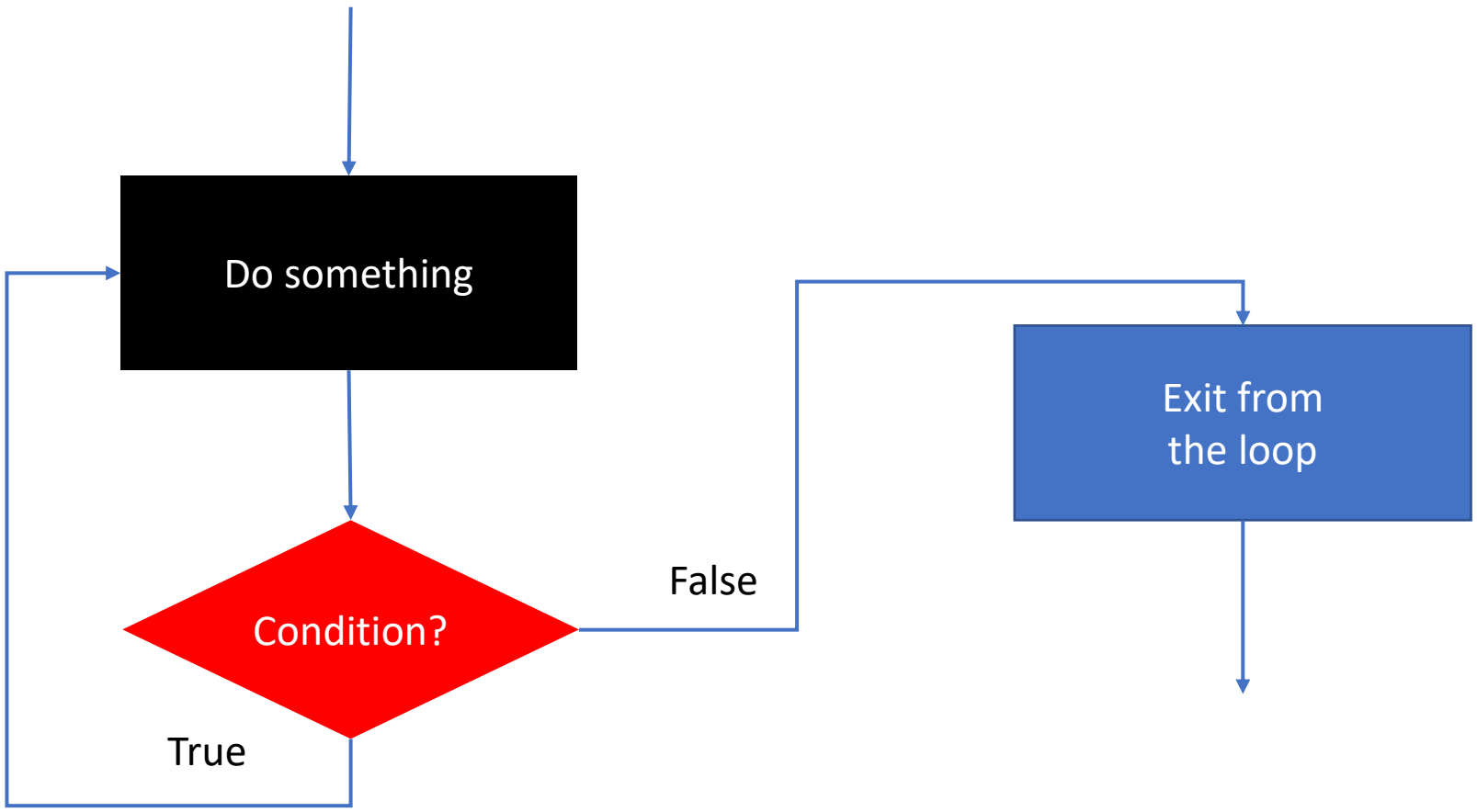
- Flow chart (while)



UCLA

Review on `for`, `while`, and `do while`

- Flow chart (do while)



UCLA

Random Numbers

Syntax, Libraries, and Usage

rand() <cstdlib>

- `rand()` generates a (pseudo-) random integer,
  - Between `0` and `RAND_MAX`
    - RAND_MAX depends on the library
    - But at least 0x7fff = $2^{15} - 1 = 32767$ on any stand library implementation
  - Using a "seed"

- Using the same seed, `rand()` will generate the same sequence of random numbers
  - More on this later

UCLA

Using `rand()` – Floating Point numbers

- Since it generates an `int` between `0` and `RAND_MAX`, you can generate a random number between `0` and `X` (inclusive) by

  - `rand_num = (rand()*1./RAND_MAX)*X;`

    Type Conversion

    Normalized to 0 ~ 1

  - Or,

  - `rand_num = (static_cast<double>(rand())/RAND_MAX)*X;`

    which is preferable

UCLA

Using rand() – Floating Point numbers

- If you want a real number between A and B,
  - The size of interval = (B – A)
  - Starts from A

- `rand_num = ((rand()*1./RAND_MAX)*(B-A) + A;`

Between 0 and (B-A)

UCLA

Using `rand()` – Integers

- For integers, we can use the % (mod/remainder) operator

- Random int between 0 and N (inclusive):
  - `n = rand()%(N+1)`
  - The remainder is always between 0 and N
    - Note that, there are **N+1** different integers in [0,N]

- Random int between A and B:

  = A + (random int between 0 and (B-A))
  - `n = A + rand() % (B-A+1)`

UCLA

A Seed for `rand()`

- The function `rand()` generates a sequence of random numbers using a seed
- The seed can be set by `srand(some_number);`
- Ex) `some_number` == 1 → `rand()` gives
  - 41
  - 18467
  - 6334
  - 26500
  - 19169
  - 15724 …

  - Exercise: Use rand() several times without srand() commands, and verify that you get the sequence above

UCLA

Seed for `rand()`

- With the same seed, you'll have the same sequence of random numbers

- So, to get a random-like numbers (pseudo-random numbers), use different seed every time you run the code

- Commonly used trick: use a ***current time*** as a seed
  - `time(nullptr)` in `<ctime>` library returns the current time in **seconds**, since *00:00, Jan 1 1970 UTC*
  - Ex) `time(nullptr) == 1643894411` when this slide was created

UCLA

Summary

- #include `<cstdlib>` for `rand()` and `srand()`
- #include `<ctime>` for `time()`
- Use `time(nullptr)` to generate a seed:
  - `srand(time(nullptr));`
  - And then use `rand()`
- Intervals:
  - `r_double = (rand()*1./RAND_MAX)*(B-A) + A;`
  - `r_int = A + rand()%(B-A+1);`

UCLA

Functions

Basic Syntax and Terminology

Passing by Parameter Vs. Passing by Value

Functions

- Functions can be
  - Declared and then defined later
    - If you declare the function first (not define it right now) then need ;
  - Defined when it is declared (so both are done at the same time)

- The declaration determines a “signature” of the function
  - [Return Type] [Function Name] [Input Parameters] (and other options later)

```
double find_max(double a, double b);
```

UCLA

Functions

- The declaration determines a “signature” of the function
  - [Return Type] [Function Name] [Input Parameters] (and other options later)

```
double find_max(double a, double b);
```

- Return Type
  - The type of the expression returned by the function
  - If the function returns nothing, can be `void`
- Function Name
  - Name of the function; cannot be a reserved word, the same naming rule applies with variables
- Input Parameters
  - The objects passed to the function
  - Number of arguments can be 0, 1, or more
  - Still need parentheses () even when the function gets zero parameters

UCLA

Functions – Examples

- A function returning the maximum of two numbers

```
double find_max(double a, double b) {
        return (a > b) ? a : b;
}
```

Recall: The ternary operator

```
A ? B : C
```

is (almost) equivalent to

```
if (A) { B; }
else {C;}
```

- A ***procedure*** is a function returning `void`
  - Which means, it doesn't return anything
  - Example: a function printing the max to the console

```
void print_max(double a, double b) {
    cout << "The max of " << a << " and " << b
         << " is " << find_max(a, b);
}
```

Possible if `find_max` is ***declared*** before this expression

Q: [`true` or `false`?]
Using "`return`" keyword in a ***procedure*** results in a syntax error.

UCLA

Functions – Examples

- A function that inputs several different types of arguments

```
void func(unsigned int i, string str) {
        cout << str[i];
}
```

Here `i` cannot be negative

- A ***predicate*** function is a function returning `bool`
  - The following function checks whether the first letter of the string is capitalized or not

```
bool isCapitalized(string str) {
    if ('A' <= str[0] && str[0] <= 'Z') {
        return true;
    }else {
        return false;
    }
}
```

True if `str[0]` is between 'A' and 'Z' (otherwise it is not a capital letter)

UCLA

Passing Parameters by Reference

- The default behavior of passing parameters to a function is to "pass by copy"
  - Also called "pass by value"
- Parameters are copied to a local variable

```
bool isPalindrome(string str) {
        // ftn body ...
}

int main() {
    string huge_str = "ABC..xyz..123...\n\n...";
    if (isPalindrome(huge_str)) {
        // ...
    }
}
```

The string variable `huge_str` is copied to a local string variable `str`

- If the parameter has huge size (in terms of memory), then copying it is inefficient

UCLA

Passing Parameters by Reference

- Solution to this inefficiency → Use a reference!
- Recall: a reference is just another name of the variable, so unnecessary copies don't occur

```
bool isPalindrome(string& str) {
        // ftn body ...
}

int main() {
    string huge_str = "ABC..xyz..123...\n\n...";
    if (isPalindrome(huge_str)) {
        // ...
    }
}
```

str is now "*a reference to a string*" (type name) and it refers to huge_str

- However, as we've seen before, this is dangerous since str can be modified in the function, and we don't want this change to affect the original huge_str

UCLA

Passing Parameters by **Const** Reference

- Solution to this danger → Use a **const** reference!
- A const reference is a reference whose value cannot be modified (so it's safe)

```
bool isPalindrome(const string& str) {
        // ftn body ...
}

int main() {
    string huge_str = "ABC..xyz..123...\n\n...";
    if (isPalindrome(huge_str)) {
        // ...
    }
}
```

str is now "*a reference to a **const** string*" (type name) and it refers to huge_str

- Here, we pass a huge string object in a **safe** and **efficient** way
  - No unnecessary copies occurred
  - No changes in the original object

UCLA

Passing Parameters by Reference

- Passing by non-const reference can be used to change multiple parameters in a single function evaluation
- Say we have three variables a, b, c and want to increment them by two

```
int a = 2, b = 3, c = 4;
cout << a << ' ' << b << ' ' << c << endl;
add2(a, b, c);
cout << a << ' ' << b << ' ' << c << endl;
```

Desired output:

```
2 3 4
4 5 6
```

- What should be ***the signature of add2***?

```
void add2(int& x, int& y, int& z)
```

- Implementation?

```
{ x+=2; y+=2; z+=2; }
```

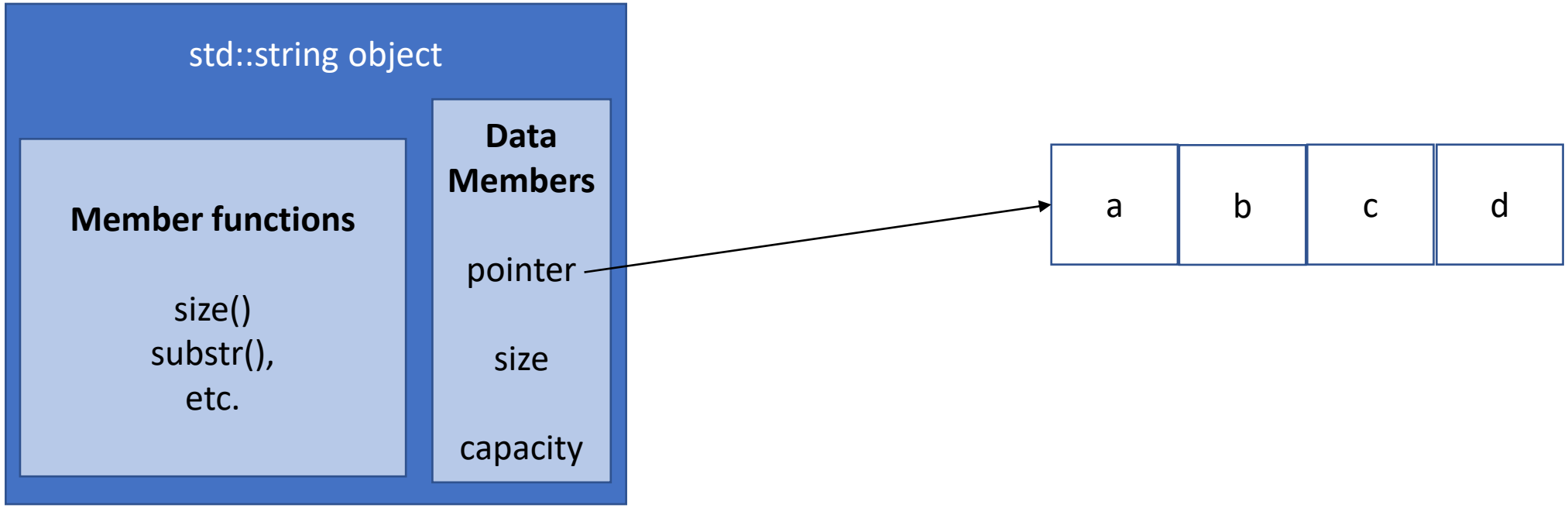
UCLA

C++ Classes

Concepts, Examples, and Exercise Problems

Classes

- Roughly speaking, a class is a user-defined type that contains a collection of data members with other features (methods)
- e.g. “std::string” has data members like
  - the pointer to the char array (actual string data)
  - size and capacity variables, and some other members
  - useful member functions like size(), substr(), and operators([], ==, <, +, etc.)



Classes

- Default accessibility of data members/functions is *private* for classes
  - On the other hand, a very similar data structure, “struct” has *public* members by default

- To access the members, use the “.”(dot) operator

```
e.g.    string str = "ABCDE";
        int len = str.length();
```

- Initialization is done by a “constructor”

```
class B {
public:
    void b() const;
    B();
};
```

```
int main() {
    B b_object;
}
```

Calling the default constructor here

UCLA

Classes

- If a class has several data members, a proper initialization may be important

```
class B {
public:
    string name;
    double salary;
    int age;
    void b() const;
    B();
    //(another c'tor)
};
```

```
int main() {
    B b_object;
    B John("John Doe", 60000, 25);
}
```

Calling the default constructor here.
Setting the name = “”, salary = 0.0 and age = 0 in the default constructor can be a good initialization

Calling another type of constructor here
(not declared in the class interface yet)

What’s the correct signature of this constructor?

```
B(string _name, double _salary, int _age);
```

UCLA

Classes

- Always use the ***constructor initializer list*** for initialization

```
class B {
public:
    string name;
    double salary;
    int age;
    void b() const;
    B();
};
```

```
B(string _name, double _salary, int _age)
     : name(_name), salary(_salary), age(_age) {}
```

```
int main() {
    B b_object;
    B John("John Doe", 60000, 25);
}
```

The constructor's **body** is empty in this case

UCLA

Vectors, Arrays, and Pointers

Basic Syntax

Comparisons Between Vector and Arrays, and Arrays and Pointers

Pointer Arithmetic

Vectors

- Vector is a special type of a class ("templated class") that can be considered an **array** of some other class type, with useful member functions
  - Recall that a "string" can be thought as an array of "char" variables, with useful member functions such as length, substr, etc.
- Must include <vector> library to use vectors
- Ex) Vector of `ints`
  - `vector<int> vint = { 2020, 5, 19 }; // contains 2020, 5, 19 in this order`
  - Access: subscript operator []
    - `cout << vint[0] << ", " << vint[1] << ", " << vint[2] << endl;`
- The vector class has many useful constructors
  - Creating a size "N" vector: `vector<int> vint2(N);`
  - Creating a size "N" vector and initialize all with "val": `vector<int> vint3(N, val);`
  - Creating a copy of other vector(copy constructor): `vector<int> vint4(vint);`

UCLA

Vectors

- What is the output of the code on the right?

- Ans: AAAAA
- Tvec is a vector of five “T” objects, where each of “T” objects is default-constructed

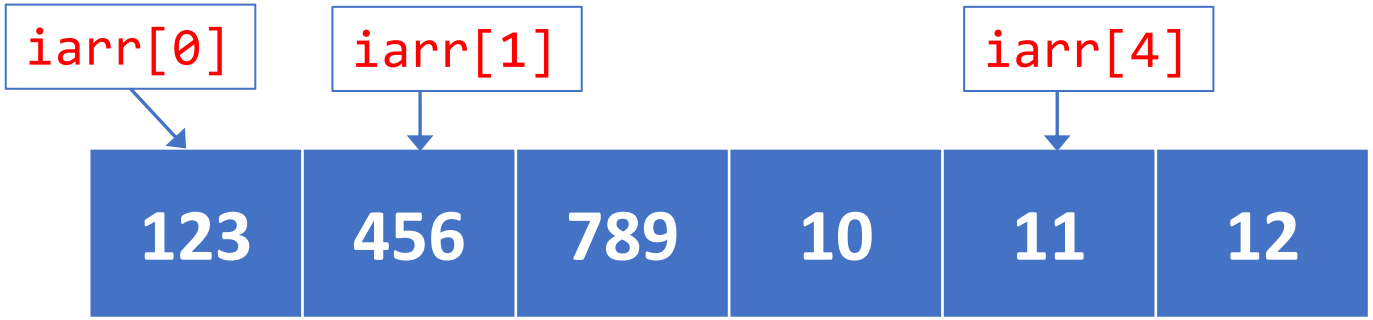
```
#include<iostream>
#include <vector>
using namespace std;
class T {
public:
    T() { cout << "A"; }
    T(int i) { cout << i; }
};
int main()
{
    vector<T> Tvec(5);

    return 0;
}
```

UCLA

Vectors and Arrays

- Vectors are like arrays, but with special features in addition
- In fact, (static) arrays are the most basic objects among “array-like” stuffs
- An int array of size 6 may look like (in the memory):



- “iarr” is the name of the array object
- This “iarr” actually “points to” the address of the part of the memory that stores the value 123 (or, iarr[0])
- Accessing the k-th element of iarr → iarr[k] (subscript operator)

Vectors and Arrays

- But (static) arrays cannot change in its size, and there are no member functions for array objects
  - It's why I called it the *most basic* "array-like" type
  - No push_back, insert, erase, etc.

- There is another type of arrays, "dynamic arrays" whose size can be changed dynamically
  - But not covered in this course, you will see them again in PIC 10B
  - Also this object is pretty difficult to deal with

- Vectors are much easier to handle, so in most cases you can just use vectors
  - Vectors = Arrays + Useful Features

UCLA

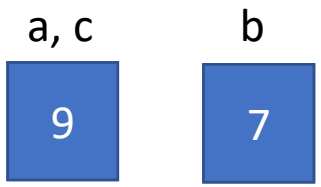
Pointers

- Recall that a reference to a variable is just “another name” of the other

```
class T {
        // ...
};
T tvar;                 // variable of type T
T& ref_tvar = tvar;     // a reference to a type T variable
```

- And they both refer to the same “address” in the memory

```
int a = 5;
int b = a;
int& c = a;
cout << a << ' ' << b << ' ' << c << endl;
b = 7;
cout << a << ' ' << b << ' ' << c << endl;
c = 9;
cout << a << ' ' << b << ' ' << c << endl;
```



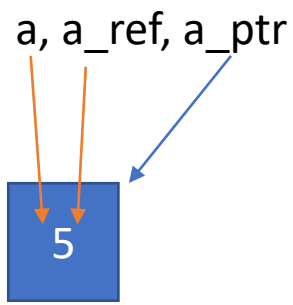
UCLA

Pointers

- So, a reference, in fact, refers to the value in some address
- Pointers are used to “point” to those address

```
int a = 5;
int& a_ref = a;
int* a_ptr = &a; // address of a
cout << a << ' ' << a_ref << ' ' << *a_ptr << endl;
a_ref = 7;
cout << a << ' ' << a_ref << ' ' << *a_ptr << endl;
*a_ptr = 9;
cout << a << ' ' << a_ref << ' ' << *a_ptr << endl;
```

```
5 5 5
7 7 7
9 9 9
```



- Note that, they store the “address” so you can’t do something like

```
a_ptr = a;
```

a value of type "int" cannot be assigned to an entity of type "int *"

Search Online

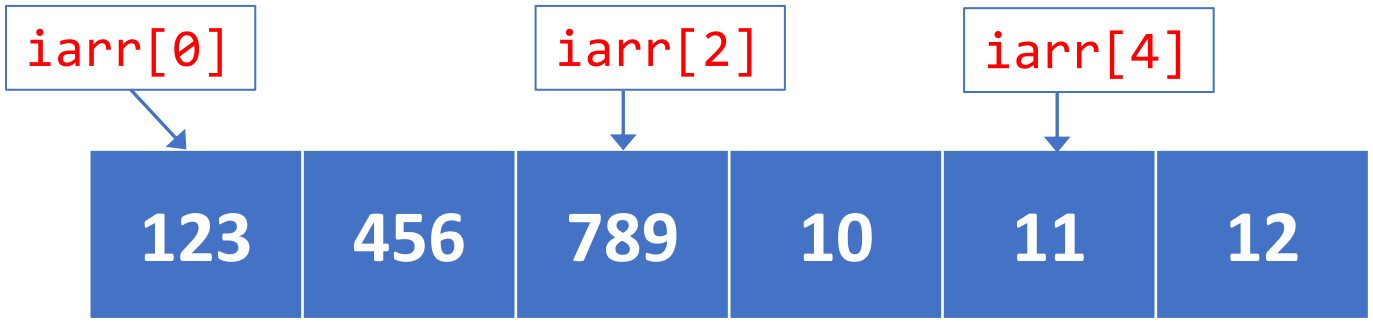
UCLA

Pointer Arithmetic

- To access the value stored in the address where your pointer points, use the ***dereference operator*** (*)

```
int a = 5;
int* a_ptr = &a; // address of a
*a_ptr = 9;   // dereference to access the value
cout << a << ' ' << *a_ptr << endl; // again, dereference to access the value
```

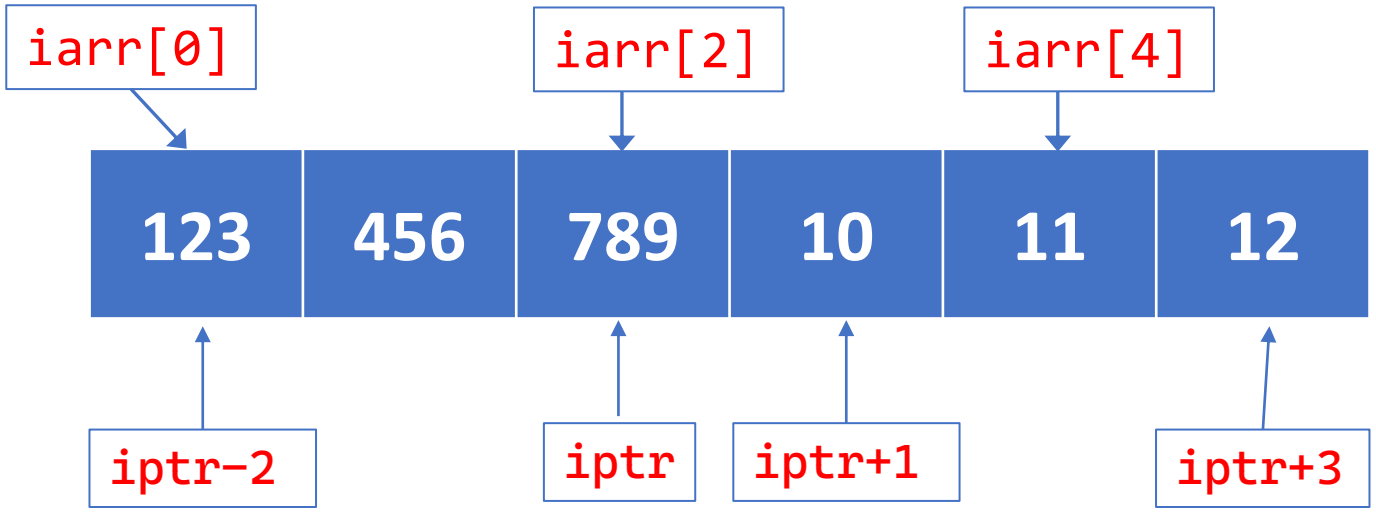
- You can also "move around" the address using "pointer arithmetic"
- Let's say you have an `int` array of size 6:



UCLA

Pointer Arithmetic

- An `int` array of size 6 may look like (in the memory):




```
int* iptr = &(iarr[2]);
// &(2nd elt of iarr)
```

- Let `iptr` be a pointer pointing to the 2nd element of the array, then you can move around by adding/subtracting numbers from its address
- This is called "pointer arithmetic" (just like "integer arithmetic" but no *,/,%, etc.)
  - +, -, and ++, -- are allowed
  - Additionally, comparisons (==, <, >) are also available

Arrays vs Pointers

```
size of a: 20
size of aptr: 8
```

- Arrays "decay" to pointers if it needs to be
- But still, they are DIFFERENT!

```
int a[] = { 1,2,3,4,5 };
int* aptr = a;
cout << "size of a: " << sizeof(a) << endl
<< "size of aptr: " << sizeof(aptr) << endl;
```

- You can compute the length of the array using "`sizeof(a)/sizeof(int)`"
- But then why do you still need to pass the array size to a function? (recall HW 5, for example)

Exercise 1: Equals

(This is essentially Exercise E6.8 of the book)
Write a program equals.cpp that implements the function

```
bool equals(int a[], int a_size, int b[], int b_size)
```

UCLA

Arrays vs Pointers

- It is because an array ***decays*** to a pointer when it is passed to a function
- You can never pass an array (by value), nor return an array
  - Passing arrays is not an error, but it decays to a pointer
  - Returning an array is an error!

- *Remark*: In fact, arrays can be passed by reference

```
int f(int(&a)[3]) {
    cout << "*size of this array: " << sizeof(a) << endl;
    return 0;
}
```

  - This function can get an array of size 3, and the output is

```
*size of this array: 12
```

UCLA

Arrays vs Pointers

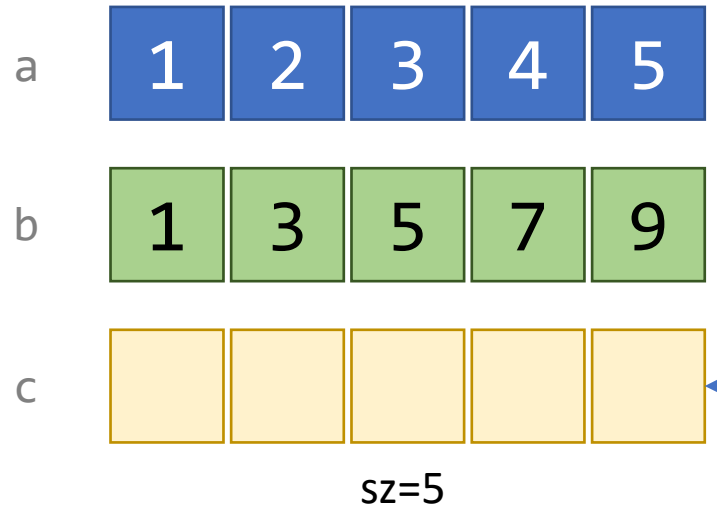
- Since the implicit conversion [array → pointer] is allowed, you can use arrays in every expression where pointers are expected

```
void add_arrays(int a[], int b[], size_t sz, int *c) {
    for (size_t i = 0; i < sz; ++i) {
        *(c++) = *(a++) + *(b++);
    }
}
```

- How does the function work?

UCLA

Arrays vs Pointers




```
void add_arrays(int a[], int b[], size_t sz, int *c) {
    for (size_t i = 0; i < sz; ++i) {
        *(c++) = *(a++) + *(b++);
    }
}
```

At least "sz" places must be reserved!
(otherwise it may corrupt your data and result in undefined behavior)

- Pointers move forward by ++
- The post-increment operator returns the original value
- Access the value stored in an address via the dereference operator *

UCLA

Exercise Problems

Only Conceptual Questions

(Coding Exercises can be found on GitHub)

Exercise (Integral Type I)

When poll is active, respond at **pollev.com/bumsukim297**

Text **BUMSUKIM297** to **37607** once to join

What is the largest unsigned integer value that can be expressed with 4 binary bits of information?

3

4

7

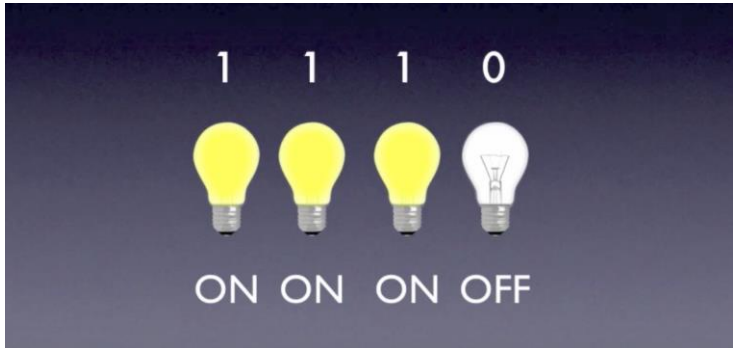
8

15

16

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**



Exercise (Integral Type I)

When poll is active, respond at **pollev.com/bumsukim297**

Text **BUMSUKIM297** to **37607** once to join

What is the largest unsigned integer value that can be expressed with 4 binary bits of information?

3

4

7

8

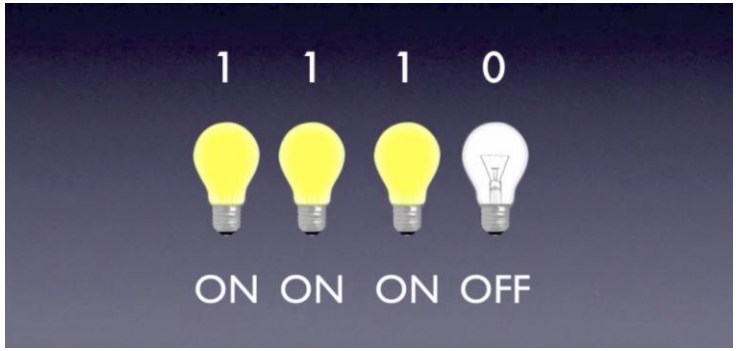
15

16

ANS: 15. We start counting from 0!

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**



Exercise (Integral Type II)

When poll is active, respond at **pollev.com/bumsukim297**

Text **BUMSUKIM297** to **37607** once to join

Can the following code overflow?

Yes, it may overflow when x is some special value.

No, it is safe.

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

```
#include <iostream>
using namespace std;
int main() {
    int x = INT_MIN;
    cout << x << endl;
    cout << -x << endl;
    return 0;
}
```

Exercise (Integral Type II)

When poll is active, respond at **pollev.com/bumsukim297**
Text **BUMSUKIM297** to **37607** once to join

Can the following code overflow?

Yes, it may overflow when x is some special value.

No, it is safe.

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

```
#include <iostream>
using namespace std;
int main() {
    int x = INT_MIN;
    cout << x << endl;
    cout << -x << endl;
    return 0;
}
```

Answer: Yes.
-INT_MIN is greater than INT_MAX!

Exercise (Integral Type III)

```
int n = 4.567; // what is the value of n?
```

When poll is active, respond at **pollev.com/bumsukim297**

Text **BUMSUKIM297** to **37607** once to join

Value of n?

- 4 (rounded down)
- 5 (rounded to the nearest whole number)
- 5 (rounded up)
- Compile Error

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Exercise (Integral Type III)

```
int n = 4.567; // what is the value of n?
```

When poll is active, respond at **pollev.com/bumsukim297**

Text **BUMSUKIM297** to **37607** once to join

Value of n?

- 4 (rounded down)
- 5 (rounded to the nearest whole number)
- 5 (rounded up)
- Compile Error

Ans: 4 (rounded down)

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Exercise (Variables)

- Q) Which of the following are literals of type `float`? (Choose all correct choices.)

  - ◯ `23.45`
  - ◯ `23.45f`
  - ◯ `23.45F`
  - ◯ `23E0`
  - ◯ `23E0f`
  - ◯ `23e0F`
  - ◯ None of the above.

UCLA

Exercise (Variables)

- Q) Which of the following are literals of type `float`? (Choose all correct choices.)

  - ◯ `23.45`
  - ◯ `23.45f`
  - ◯ `23.45F`
  - ◯ `23E0`
  - ◯ `23E0f`
  - ◯ `23e0F`
  - ◯ None of the above.

A) B, C, E, F

UCLA

Exercise (Programming Concept)

- Q) Which of the following choices most accurately define the term *hard-coding*?

  A. The practice of coding under hard, rigid principles.
  B. Putting something into your code, instead of taking it in as an input.
  C. Coding in an unnecessary complicated, hard way when an easy alternative is available.
  D. None of the above.

UCLA

Exercise (Programming Concept)

- Q) Which of the following choices most accurately define the term *hard-coding*?

  A. The practice of coding under hard, rigid principles.
  B. Putting something into your code, instead of taking it in as an input.
  C. Coding in an unnecessary complicated, hard way when an easy alternative is available.
  D. None of the above.

- A) B.
- Avoid hard-coding unless it's necessary

UCLA

Exercise (Coding Practice)

- Suppose that `runTime` is a double variable storing the number of seconds (down to the nearest hundredth of a second) it takes someone to complete a marathon.

- Consider the code below that is to compute their time, rounded to the nearest second, in the format of hours, minutes, and seconds (seconds and minutes should not exceed 60).
  Does the code work?

```
int a = static_cast<int>(runTime + 0.5);
int hours = a / 3600;
int minutes = (a % 3600) / 60;
int seconds = a % 60;

cout << "Rounded run time: "
       << hours << " hour(s), " << minutes
       << " minute(s), " << seconds << " second(s)";
```

A) No, there is a logical error and the result is wrong.
B) Yes, but the coding style needs improvement
C) Yes, and the style is fine.

Exercise (Coding Practice)

- Suppose that `runTime` is a double variable storing the number of seconds (down to the nearest hundredth of a second) it takes someone to complete a marathon.

Ans: B.
Use descriptive variable name instead of “a”, for instance, “runTimeInSecond”

- Consider the code below that is to compute their time, rounded to the nearest second, in the format of hours, minutes, and seconds (seconds and minutes should not exceed 60).
  Does the code work?

```
int a = static_cast<int>(runTime + 0.5);
int hours = a / 3600;
int minutes = (a % 3600) / 60;
int seconds = a % 60;

cout << "Rounded run time: "
       << hours << " hour(s), " << minutes
       << " minute(s), " << seconds << " second(s)";
```

A) No, there is a logical error and the result is wrong.
B) Yes, but the coding style needs improvement
C) Yes, and the style is fine.

Exercise (String Manipulation I)

- Q) What is the output for the following code?

```
string s1, s2, s3;
s1 = "PIC";
s2 = "10A";
s3 = s1 + s2;
s3 += "\n";
cout << s2.length() << " " << s3.length();
```

UCLA

Exercise (String Manipulation I)

- Q) What is the output for the following code?

```
string s1, s2, s3;
s1 = "PIC";
s2 = "10A";
s3 = s1 + s2;
s3 += "\n";
cout << s2.length() << " " << s3.length();
```

- A) 3 7

```
string s1, s2, s3;
s1 = "PIC";    // length == 3
s2 = "10A";
s3 = s1 + s2; // s3 == "PIC10A"    (length == 6)
s3 += "\n";    // s3 == "PIC10A\n" (length == 7)
cout << s2.length() << " " << s3.length();
```

UCLA

Exercise (String Manipulation II)

- Q) Consider the following program.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  string s = "a more perfect union";
  cout << s[1];
  cout << s[3];
  cout << s[5] << "\n";
}
```

What is the output?

A. oe
B. a m
C. amo
D. Neither of the above

Exercise (String Manipulation II)

- Q) Consider the following program.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  string s = "a more perfect union";
  cout << s[1];
  cout << s[3];
  cout << s[5] << "\n";
}
```

What is the output?

A. oe
B. a m
C. amo
D. Neither of the above

A) A.
C++ indexing always starts from 0

UCLA

Exercise (Swap)

- Q) Suppose a,b,c are int type variables that have been initialized appropriately. Reorder the following lines to swap the values of a and b. The value that c stores doesn't matter.

```
(1) a = b;
(2) b = c;
(3) c = a;
```

Exercise (Swap)

- Q) Suppose a,b,c are int type variables that have been initialized appropriately. Reorder the following lines to swap the values of a and b. The value that c stores doesn't matter.

```
(1) a = b;
(2) b = c;
(3) c = a;
```

- A) (3)-(1)-(2)

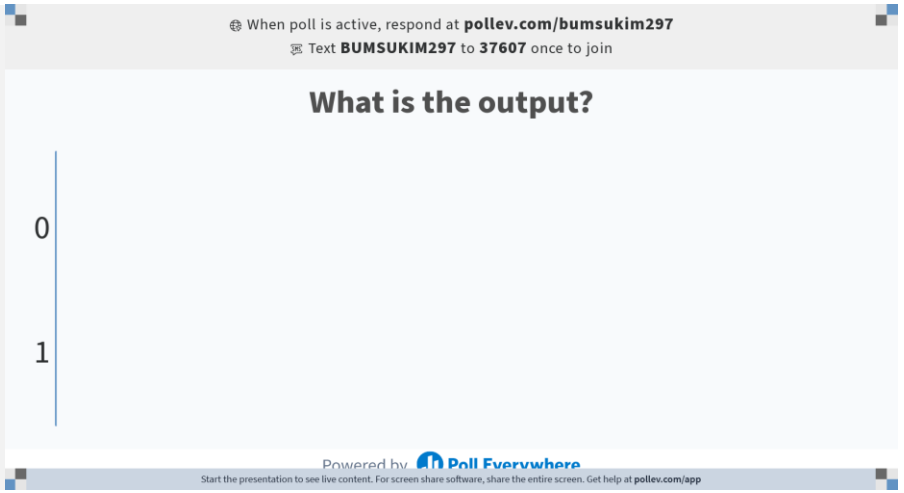
UCLA

Exercise (Boolean Algebra)

2. Consider the following program.

```
#include <iostream>
using namespace std;

int main() {
  bool b1 = true, b2 = false;
  cout << ( !(b1 && !b2) || b2 ) << endl;
}
```



What is the output?

A. 0

B. 1

- Recall that `true` outputs 1, and `false` outputs 0 on the console
  - (Can be strings "true" and "false," respectively, if you use the option `std::boolalpha`)

Exercise (Boolean Algebra)

```
bool b1 = true, b2 = false;
```

```
!(b1 && !b2) || b2
```

True

True

False

&& (AND)

True

! (not)

OR

False

False

UCLA

Exercise (Control Flow I)

7. Consider the following program, which is poorly indented on purpose.

```
#include <iostream>
using namespace std;

int main() {
if (true)
cout << "a";
else
cout << "b";
cout << "c";
}
```

What is the output?

A. a B. b C. c D. ab E. ac F. bc G. abc

UCLA

Exercise (Control Flow I)

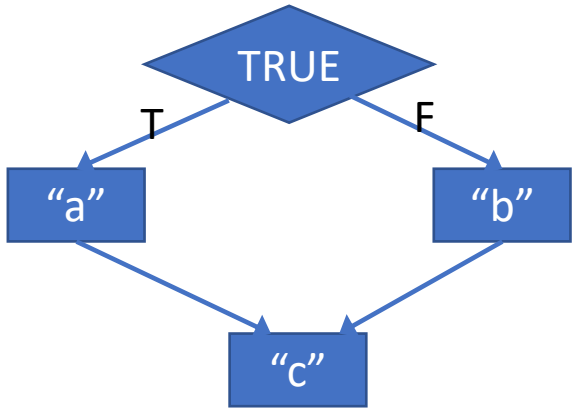
7. Consider the following program, which is poorly indented on purpose.

```
#include <iostream>
using namespace std;

int main() {
if (true)
cout << "a";
else
cout << "b";
cout << "c";
}
```

What is the output?

A. a B. b C. c D. ab E. ac F. bc G. abc



UCLA

Exercise (Control Flow II)

2. Consider the following program.

```
#include <iostream>
using namespace std;

int main() {
  int iter = 0;
  int n;
  cin >> n;
  while(iter<n) {
    iter += 2;
    cout << iter << endl;
  }
}
```

Which of the following choices most accurately describe the functionality?

A. Print all positive even numbers less than the input.
B. Print all positive even numbers less than or equal to the input.
C. Print all positive even numbers less than or equal to the input + 1.
D. Print all positive even numbers less than or equal to the input + 2.
E. None of the above.

UCLA

Exercise (Control Flow II)

2. Consider the following program.

```
#include <iostream>
using namespace std;

int main() {
  int iter = 0;
  int n;
  cin >> n;
  while(iter<n) {
    iter += 2;
    cout << iter << endl;
  }
}
```

Which of the following choices most accurately describe the functionality?

A. Print all positive even numbers less than the input.
B. Print all positive even numbers less than or equal to the input.
C. Print all positive even numbers less than or equal to the input + 1.
D. Print all positive even numbers less than or equal to the input + 2.
E. None of the above.

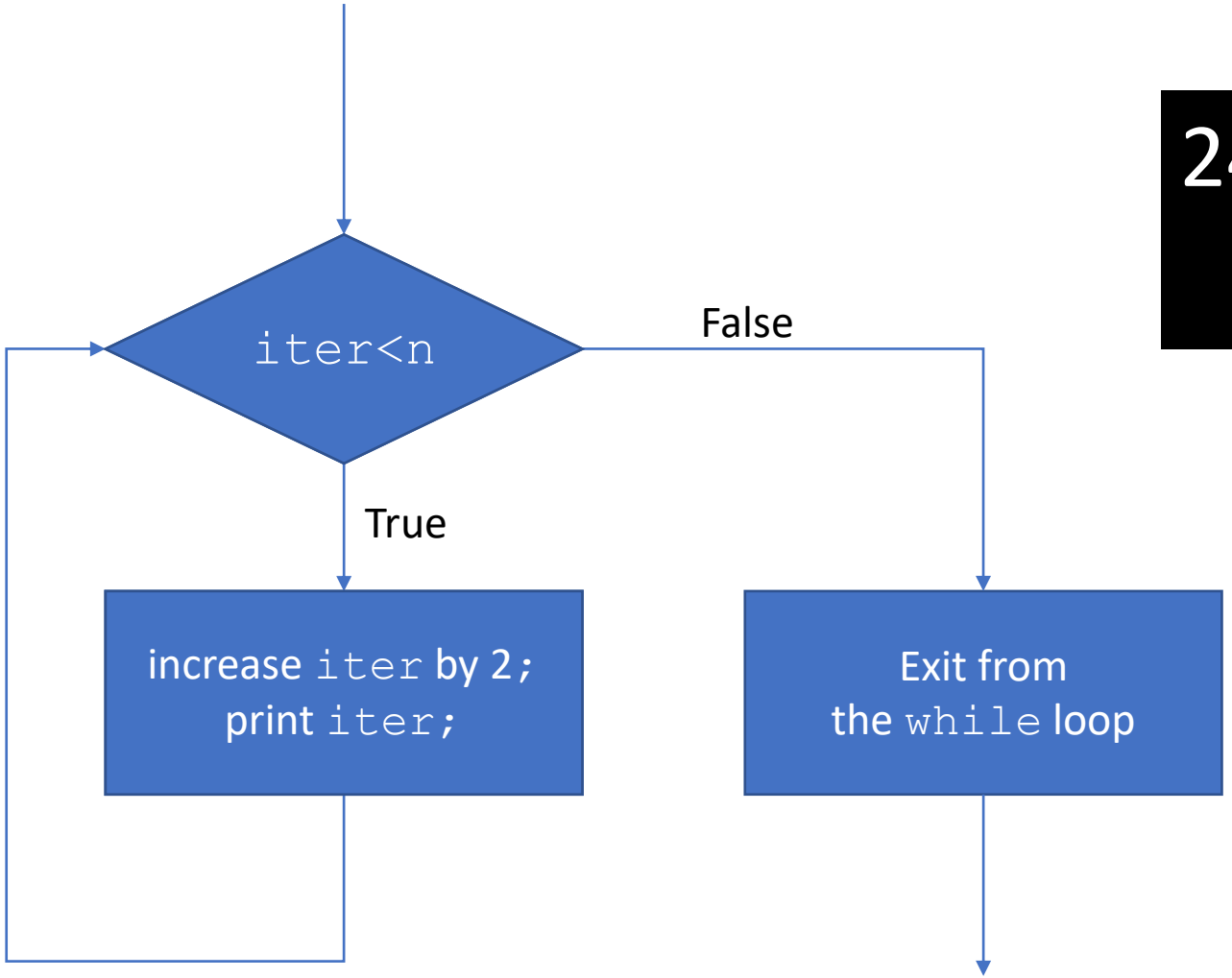
- Answer: C
- Execute the statements in the loop only when `iter < n`, but `iter` immediately becomes `iter+2`
- Consider the following examples →

UCLA

Exercise (Control Flow II)

- **Ex1:** `n=4`

```
1. iter=0
2. iter(0)<4?
3. iter←2
4. print 2
5. iter(2)<4?
6. iter←4
7. print 4
8. iter(4)<4?
9. exit
```

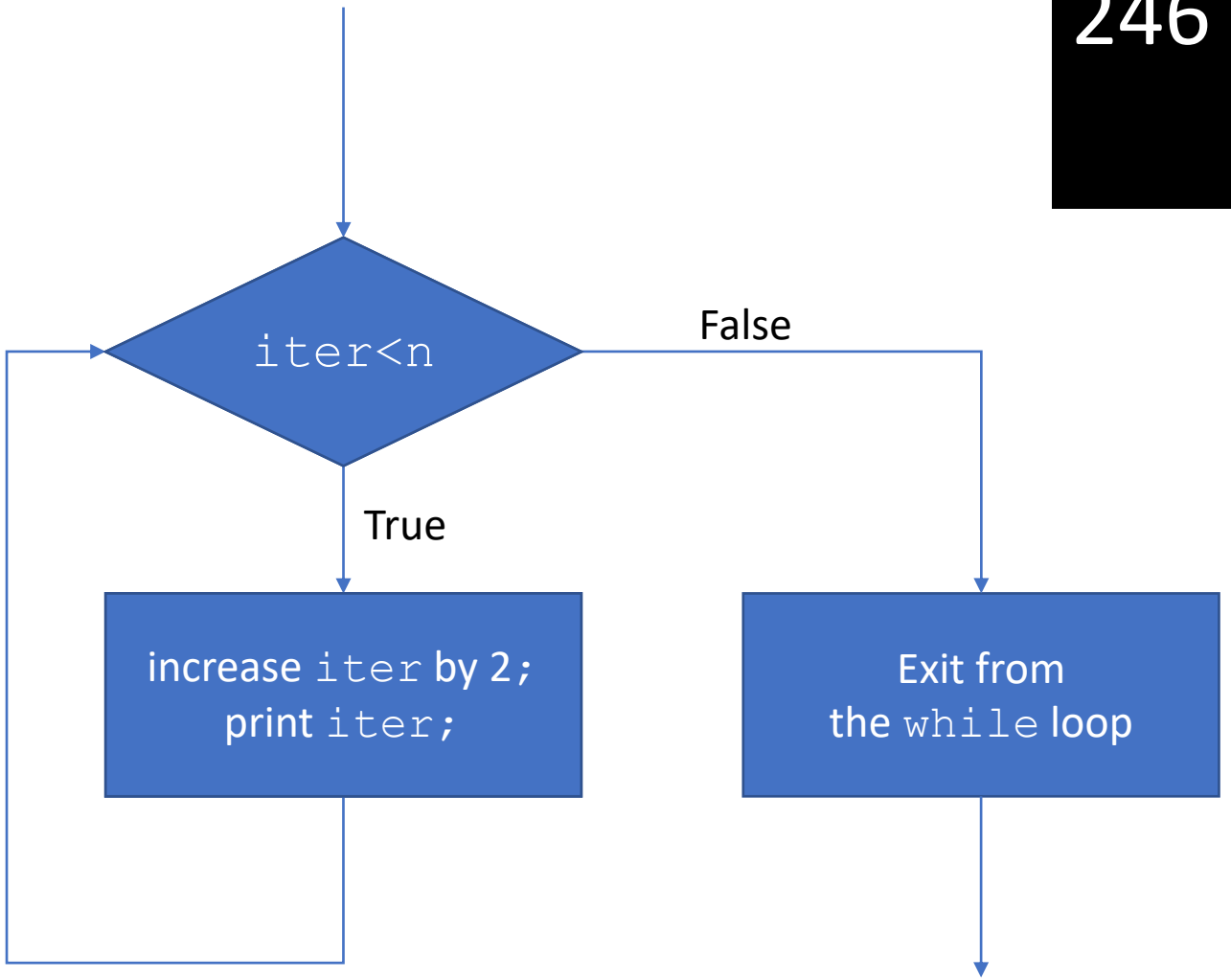


24

Exercise (Control Flow II)

- Ex2: `n=5`

```
1. iter=0
2. iter(0)<5?
3. iter←2
4. print 2
5. iter(2)<5?
6. iter←4
7. print 4
8. iter(4)<5?
9. iter←6
10.print 6
11.exit
```



246

Exercise (Control Flow III)

- Predict the output of the code below:

```
cout << "start" << '\n';
for (int i = 1; i < 4; ++i) {
    for (int j = 1; j < i; ++j) {
        cout << "*";
    }
    cout << '\n';
}
cout << "end";
```

UCLA

Exercise (Control Flow III)

- Predict the output of the code below:

```
cout << "start" << '\n';
for (int i = 1; i < 4; ++i) {
    for (int j = 1; j < i; ++j) {
        cout << "*";
    }
    cout << '\n';
}
cout << "end";
```

```
start

*
**
end
```

UCLA

Exercise (Control Flow IV)

- Describe the value of `j` in terms of `i` after the for loop:

```
int i = (some positive integer);
int j;
for (j = i; (j / 10) > 0; j /= 10);
```

- Hint) When `i = 4567`, what is `j`?

UCLA

Exercise (Control Flow IV)

- Describe the value of `j` in terms of `i` after the for loop:

```
int i = (some positive integer);
int j;
for (j = i; (j / 10) > 0; j /= 10);
```

- Hint) When `i = 4567`, what is `j`?

- Ans) `j` represents the first digit of `i`.
  For instance,
  when `i = 4567`, `j = 4`.

UCLA

Exercise (Vectors I)

- Suppose that vec is a std::vector<int>, represented by
  vec == { 3, 8, -6, 0, -8, 12, 1 }.
- Determine the value of x after the code below has executed.

```
int x = vec[0];
for (auto i : vec) {
        x = ((x < i) ? i : x);
}
```

UCLA

Exercise (Vectors I)

- Suppose that vec is a std::vector<int>, represented by
  vec == { 3, 8, -6, 0, -8, 12, 1 }.
- Determine the value of x after the code below has executed.

```
int x = vec[0];
for (auto i : vec) {
        x = ((x < i) ? i : x);
}
```

Answer: x is 12 (*max* of vec)

UCLA

Exercise (Vectors II)

- Q. How do you define a vector to store salaries of 10 employees?
- Select one:
  - a. vector<double> employee_salary(10);
  - b. <double>vector employee_salary(10);
  - c. vector employee_salary(10);
  - d. double employee_salary(10);

UCLA

Exercise (Vectors II)

- Q. How do you define a vector to store salaries of 10 employees?
- Select one:
  - a. vector<double> employee_salary(10);
  - b. <double>vector employee_salary(10);
  - c. vector employee_salary(10);
  - d. double employee_salary(10);

- Ans: a
  - Type of the vector is `vector<double>`
  - The size is passed to a constructor to this vector classvector<double>

UCLA

Exercise (Vectors III)

- Q. What is the output of the following code snippet?
- Select one:
  - a. 0 1 2 3 4 5 6 7 8 9
  - b. 0 2 4 6 8
  - c. 1 3 5 7 9
  - d. 0 1 2 3 4

```
vector<int> myvector;
int n;
for (int i = 0; i < 10; i++)
{
    myvector.push_back(i);
    n = i % 2;
    if (n == 0)
    {
        myvector.pop_back();
    }
}
for (int i = 0; i < myvector.size(); i++)
{
    cout << myvector[i] << " ";
}
```

UCLA

Exercise (Vectors III)

- Q. What is the output of the following code snippet?
- Select one:
  - a. 0 1 2 3 4 5 6 7 8 9
  - b. 0 2 4 6 8
  - c. 1 3 5 7 9
  - d. 0 1 2 3 4

```
vector<int> myvector;
int n;
for (int i = 0; i < 10; i++)
{
    myvector.push_back(i);
    n = i % 2;
    if (n == 0)
    {
        myvector.pop_back();
    }
}
for (int i = 0; i < myvector.size(); i++)
{
    cout << myvector[i] << " ";
}
```

- Ans: c
  - 1 →
  - 1 2 → 1 →
  - 1 3 →
  - 1 3 4 → 1 3 →
  - 1 3 5 →
  - 1 3 5 6 → 1 3 5 → …

*pop_back( ) called when i is even

UCLA

Exercise (Vectors IV)

- Q. Let print() be a member function of class Player. Complete the following code to print all elements in the vector backwards (starting from the last element and ending with the first one).

```
vector<Player> mylist;
// add elements to mylist
int i = 0;
for (___A___; ___B___; ___C___)
{
    ___D___;
}
```

- A: `i = mylist.size() - 1;`
- B: `i >= 0;`
- C: `--i;`
- D: `(mylist[i]).print();`

UCLA

Exercise (Vectors IV)

- Q. Let print() be a member function of class Player. Complete the following code to print all elements in the vector backwards (starting from the last element and ending with the first one).

```
vector<Player> mylist;
// add elements to mylist
int i = 0;
for (___A___; ___B___; ___C___)
{
    ___D___;
}
```

- A: `i = mylist.size() - 1;`
- B: `i >= 0;`
- C: `--i;`
- D: `(mylist[i]).print();`

- for (initialization ; condition ; update){
  // Loop body
  }

UCLA

Exercise (Arrays I)

- Q. What is the output of the following code snippet?
- Select one:
  - a. 0
  - b. 1
  - c. 2
  - d. 3

```
int s = 0;
int myarray[3];
for (int i = 0; i < 3; i++)
{
   myarray[i] = s;
   s = s + i;
}
cout << myarray[2];
```

UCLA

Exercise (Arrays I)

- Q. What is the output of the following code snippet?
- Select one:
  - a. 0
  - b. 1
  - c. 2
  - d. 3

```
int s = 0;
int myarray[3];
for (int i = 0; i < 3; i++)
{
   myarray[i] = s;
   s = s + i;
}
cout << myarray[2];
```

- Ans: b
  - `myarray[0] = 0;  s = 0+0;`
  - `myarray[1] = 0;  s = 0+1;`
  - `myarray[2] = 1;  s = 1+2;`

  - Update `s` after assigning `myarray[i]`

UCLA

Exercise (Classes I)

Consider the following code.

```
#include <iostream>
using namespace std;

class T {
 public:
  int mem;
  T() { mem = 5; }
  void fn(int x) { mem += x; }
};

int main() {
  T t1;
  t1.fn(2);
  t1.fn(3);
  cout << t1.mem << endl;
}
```

The output is

A. 0.
B. 2.
C. 3.
D. 5.
E. 10.

UCLA

Exercise (Classes I)

Consider the following code.

```
#include <iostream>
using namespace std;

class T {
 public:
  int mem;
  T() { mem = 5; }
  void fn(int x) { mem += x; }
};

int main() {
  T t1;
  t1.fn(2);
  t1.fn(3);
  cout << t1.mem << endl;
}
```

The output is

A. 0.
B. 2.
C. 3.
D. 5.
E. 10.

- Ans: E (10)

UCLA

Exercise (Classes II)

Consider the following code.

```
#include <iostream>
using namespace std;

class T {
 public:
  ...
  void fn(T t);
};

int main () {
  T t1, t2;
  ...
  t1.fn(t2); //here
}
...
void T::fn(T t) { ... }
```

The line `t1.fn(t2);` can change

A. neither `t1` nor `t2`.
B. `t1` but not `t2`.
C. `t2` but not `t1`.
D. both `t1` and `t2`.

UCLA

Exercise (Classes II)

Consider the following code.

```
#include <iostream>
using namespace std;

class T {
 public:
  ...
  void fn(T t);
};

int main () {
  T t1, t2;
  ...
  t1.fn(t2); //here
}
...
void T::fn(T t) { ... }
```

The line `t1.fn(t2);` can change

A. neither `t1` nor `t2`.
B. `t1` but not `t2`.
C. `t2` but not `t1`.
D. both `t1` and `t2`.

- Ans: B
  - Note that `t2` is passed by value (copy)
  - On the other hand, the function `fn` is not marked as `const`, so it may change `t1` (its implicit parameter)

UCLA

Exercise (Classes III)

What is the output of the following code snippet?

```
class Car
{
public:
   void set_speed(double new_speed);

private:
   double speed;
};
void Car::set_speed(double new_speed)
{
   speed = new_speed;
}
int main()
{
   Car ch1;
   ch1.set_speed(144);
   cout << "Car speed: " << ch1.speed;
   return 0;
}
```

- a. Car speed: 144
- b. Car speed: 0
- c. Car speed:
- d. The code snippet does not compile.

UCLA

Exercise (Classes III)

What is the output of the following code snippet?

```
class Car
{
public:
   void set_speed(double new_speed);

private:
   double speed;
};
void Car::set_speed(double new_speed)
{
   speed = new_speed;
}
int main()
{
   Car ch1;
   ch1.set_speed(144);
   cout << "Car speed: " << ch1.speed;
   return 0;
}
```

- a. Car speed: 144
- b. Car speed: 0
- c. Car speed:
- d. The code snippet does not compile.

- Ans: d
- ch1.speed is not accessible in the main() scope.

UCLA

Exercise (Classes IV)

- Consider the classes:

```
struct A {
    A();
    void a() const;
};
class B {
    void b() const;
public:
    B();
};
```

- with the accompanying code:

```
A x;
B y;
x.a();
y.b();
```

- Which statement generates a compiler error?

UCLA

Exercise (Classes IV)

- Consider the classes:

```
struct A {
    A();
    void a() const;
};
class B {
    void b() const;
public:
    B();
};
```

- with the accompanying code:

```
A x;
B y;
x.a();
y.b();
```

B::b() is a private method!
Thus cannot be accessed outside the class.

- Which statement generates a compiler error?

UCLA